EDICIÓN XXIX

ESCUELA DE INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Revista Ciencias, Sistemas y Tecnologia

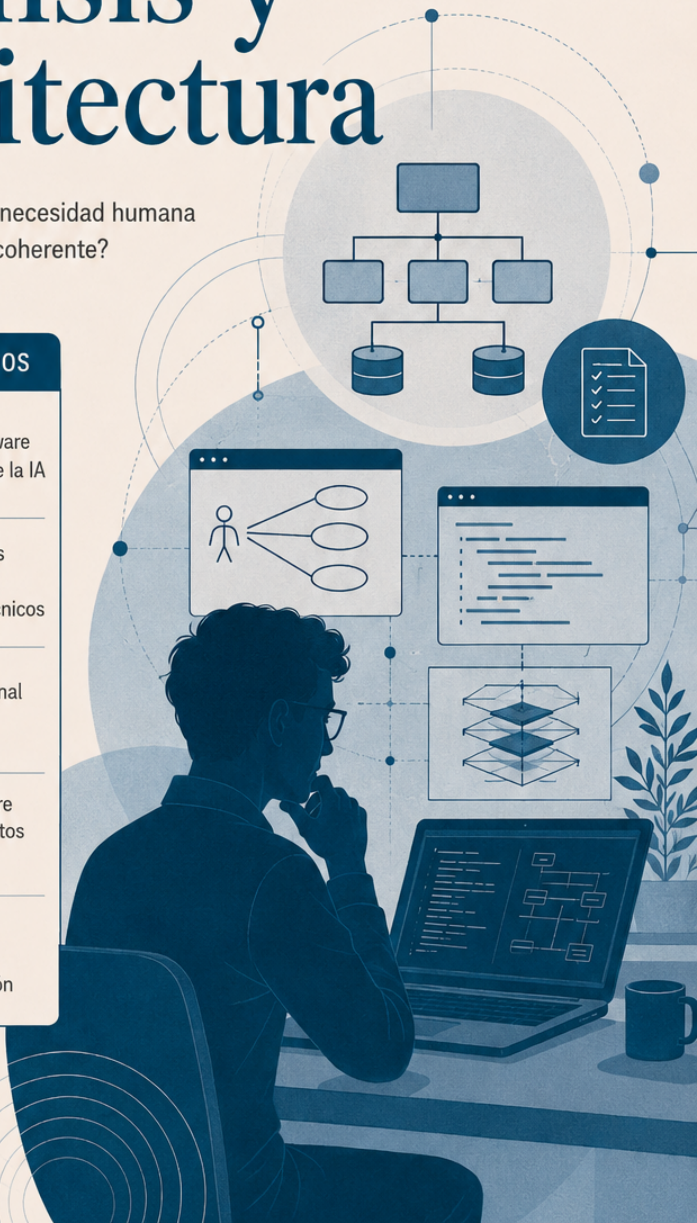
SECCIÓN 2

Análisis y arquitectura

¿Cómo se convierte una necesidad humana en una solución técnica coherente?

ARTÍCULOS DESTACADOS

- El analista de software antes y después de la IA
- De las necesidades humanas a los requerimientos técnicos
- Del análisis funcional a la arquitectura
- Comunicación entre analistas, arquitectos y programadores
- IA para analizar requerimientos: apoyo y supervisión



ENTREVISTA · PROFESIONAL

Ing. Julio Marroquín

Analista de sistemas



TEMA

Del requerimiento a la arquitectura

Ver la entrevista

¿Qué papel ocupa el analista entre el usuario, la arquitectura y el desarrollo?

Julio Marroquín lleva alrededor de diez o doce años trabajando en análisis de sistemas y describe el rol como un puente que muchas veces se subestima. El analista es la cara frente al usuario y, al mismo tiempo, la voz que lleva ese contexto hacia arquitectos y programadores. Su trabajo consiste en traducir lo que la persona solicita, lo que el proceso realmente necesita y lo que el equipo técnico puede convertir en una solución sostenible.

ANTES

Antes de utilizar asistentes de IA, ¿cómo realizaba el levantamiento y la validación de requerimientos?

El punto de partida eran reuniones extensas con usuarios y responsables del proceso. Se tomaban notas a mano, se solicitaba documentación y después se elaboraban diagramas que mostraban qué hacía el usuario, qué debía hacer el sistema y cuál sería el resultado. Era un trabajo completamente manual: hojas, lapiceros, dibujos y explicaciones sencillas para confirmar que el equipo había entendido lo mismo que la persona solicitante.

¿Qué diferencia establece entre preanálisis y análisis?

En su forma de trabajo, el preanálisis consiste en comprender el proceso y producir un primer entregable que permita conversar: un prototipo. El análisis comienza cuando ese prototipo se une con las reglas de negocio, restricciones y giro de la organización. Esa distinción evita saltar demasiado rápido a la solución técnica. Primero se hace visible lo que creemos haber entendido; después se comprueba si realmente puede convertirse en un sistema coherente.

ANTES

¿Qué parte consume más tiempo y qué habilidades son indispensables?

Entender procesos sigue siendo una de las tareas más costosas. Muchas personas desempeñan un puesto sin tener documentado el proceso completo, por lo que el analista debe reconstruirlo mediante conversación. Allí aparecen las habilidades blandas: escuchar, preguntar, ordenar y, sobre todo, encauzar la reunión. Una sesión puede convertirse fácilmente en un desahogo sobre el sistema actual; el analista debe reconocer esa frustración sin perder el objetivo de la toma de requerimientos.

ENTREVISTA · CONTINUACIÓN

Ing. Julio Marroquín

Continuación

TRANSICIÓN

¿Cuál fue la primera actividad que cambió con la llegada de la IA?

El prototipado. Antes se trabajaba con wireframes o mockups de baja fidelidad: dibujos planos, sin colores y casi sin experiencia de usuario. Con herramientas generativas, esos mismos bocetos comenzaron a convertirse rápidamente en prototipos funcionales en HTML. Para Julio el cambio fue importante porque el usuario dejó de imaginar el comportamiento y pudo abrir el prototipo en un navegador, navegarlo y reconocerse dentro de la solución. La validación pasó de describir a experimentar.

TRANSICIÓN

Cuando el cliente expresa lo que quiere, ¿cómo separa la solicitud literal de la necesidad real?

La experiencia le enseñó a volver siempre al proceso. Un usuario puede pedir reportes, biometría o funciones espectaculares porque las vio en otra aplicación, pero el sistema debe responder primero a lo que su operación necesita. Julio resume la idea con una regla práctica: automatizar procesos, no costumbres. Una práctica repetida durante años no necesariamente es un proceso bien definido ni una norma que el nuevo sistema deba conservar.

DESPUÉS

¿Qué papel tiene la documentación en esa traducción entre usuario y equipo técnico?

Aunque reconoce que la documentación suele ser un punto incómodo para muchos equipos, también aparece como una condición para comprender el proceso. Pedir al usuario documentos, representar flujos y dejar una explicación de lo entendido permite que la conversación no dependa solo de la memoria. El documento no sustituye el diálogo; lo vuelve verificable. Sirve para comparar lo solicitado con lo que arquitectos y programadores terminarán construyendo.

DESPUÉS

¿Cómo influye el presupuesto en la arquitectura que puede proponerse?

La factibilidad forma parte del análisis desde temprano. Si alguien pide recuperar una contraseña mediante lectura de retina, técnicamente puede estudiarse, pero la conversación cambia cuando se muestran el costo de implementación, mantenimiento, especialistas e infraestructura. El presupuesto obliga a priorizar. En lugar de intentar construir una versión gigantesca desde el primer día, el analista ayuda a definir qué se necesita ahora y qué puede quedar para una versión futura.

CIERRE

¿Qué permanece esencial aunque la IA pueda generar prototipos y documentación?

El criterio para comprender el proceso y la capacidad de conversar con personas. Una herramienta puede dibujar, resumir o producir HTML, pero no reemplaza la responsabilidad de descubrir qué problema vale la pena resolver. El analista sigue siendo quien mantiene alineadas intención, reglas de negocio, restricciones y expectativas. La IA acelera el artefacto; el trabajo profesional consiste en conservar el significado que ese artefacto debe representar.

IDEA CENTRAL

La IA puede acelerar el prototipo. El analista sigue siendo responsable de comprender el proceso que ese prototipo representa.

ARTÍCULO · ESTUDIANTE

Programar no es suficiente: comprender el problema antes de diseñar el sistema

Autores: Carlos Manuel Lima y Lima; Josseline Griselda Montecinos Hernández

Palabras clave: Requerimientos, usuarios finales, dominio del negocio, historias de usuario.

Introducción

Hoy en día, desarrollar software es más rápido que antes. Las herramientas de inteligencia

artificial pueden generar código, proponer soluciones e incluso ayudar a construir sistemas en

poco tiempo. Sin embargo, que un sistema funcione técnicamente no significa que resuelva el

problema para el que fue creado. Antes de programar, el ingeniero necesita comprender el

negocio, los usuarios, sus reglas y necesidades.

De aquí surge una pregunta clave: si el ingeniero puede programar una solución pero no

comprende el problema, ¿realmente está desarrollando el sistema correcto? Para evitar esto,

conocer el dominio del negocio y transformar las necesidades del usuario en requerimientos

adecuados es fundamental. En definitiva, el desarrollo no depende únicamente de escribir

código, sino de entender correctamente el problema desde el principio.

Desarrollo

Cuando un usuario necesita que los ingenieros resuelvan un problema, él ya tiene definido cuál

es su problema, porque conoce su negocio, ha interactuado con sus clientes y sabe todas sus

reglas y restricciones. Cuando transmite sus necesidades, la mayoría de ingenieros no son

expertos en el negocio; el experto es el solicitante. Pero no basta solo con escucharlo, para dar

una buena solución los ingenieros también deben conocer y comprender el negocio. La toma de

requerimientos no debe ser platicar con el cliente y escuchar que quiere un sistema para

monitorear sus ganancias. Los ingenieros deben comprender el área en la que trabaja, conocer

cuáles son sus ingresos y egresos, de qué forma obtiene sus materias primas, cómo las vende al

cliente, entre otras cosas. Solo al comprender estos procesos es posible identificar qué

información necesita realmente el sistema y cuáles son los requerimientos que deben

considerarse durante su desarrollo. El conocimiento del dominio no es un aspecto secundario durante la toma de requerimientos. El International Institute of Business Analysis establece que, al preparar una actividad de toma de

requerimientos, se debe considerar el dominio del negocio y la participación de expertos en el

área, ya que estos pueden aportar información y orientar sobre las fuentes que deben

consultarse.[1] Esto permite comprender que el trabajo del ingeniero no consiste únicamente en

recopilar lo que el usuario expresa, sino en analizar la información obtenida y transformarla

posteriormente en requerimientos y diseños.[2]

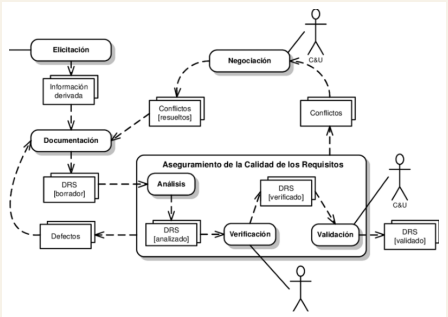


Ilustración 1. Modelo simplificado del proceso de ingeniería de requisitos. Fuente: Bernárdez, Beatriz; Amador Durán Toro y Miguel Toro (2003), consultado en ResearchGate.

ResearchGate.

Por esta razón, el conocimiento del dominio adquiere importancia durante la toma de

requerimientos, ya que permite al ingeniero comprender las reglas, procesos y restricciones que

existen dentro de un determinado entorno. No se trata únicamente de saber qué sistema quiere el

cliente, sino de comprender por qué lo necesita y cómo funciona el contexto en el que será

utilizado. A partir de esta relación entre el conocimiento del dominio y la ingeniería de

requisitos, se puede analizar hasta qué punto un ingeniero debe involucrarse en el área para la

que desarrolla una solución.

Un requisito puede parecer claro para el usuario, pero resultar ambiguo para quien debe

convertirlo en una solución técnica. Por ejemplo, solicitar “un sistema para monitorear las ganancias” no permite determinar qué

información debe considerar el sistema ni cómo debe

realizarse el cálculo. Después de analizar el negocio, este requisito podría precisar los ingresos,

egresos, costos y ventas que deben registrarse y relacionarse para obtener la información

solicitada. Este análisis permite transformar una necesidad general en un requerimiento más

preciso y útil para las siguientes etapas del desarrollo.[3]

La inteligencia artificial también apoya durante la ingeniería de requerimientos, para organizar

información, analizar historias de usuario, identificar posibles inconsistencias y proponer

elementos relacionados con el dominio. Sin embargo, estas herramientas no sustituyen la

participación del ingeniero, quien debe revisar los resultados y comprobar que la información

obtenida corresponde con las necesidades, reglas y restricciones del negocio.[3]

Conclusiones

Comprender el problema antes de desarrollar permite que el sistema responda mejor a lo que

realmente necesita el usuario. No se trata únicamente de recopilar solicitudes, sino de entender

cómo funciona el negocio, qué restricciones existen y qué situaciones deben tomarse en cuenta

para convertirlas en requerimientos útiles. El ingeniero no necesita convertirse en experto del

negocio, pero debe comprenderlo lo suficiente para traducir sus necesidades en requerimientos.

Programar sigue siendo una parte fundamental del desarrollo de software, pero por sí sola no

garantiza una buena solución si se parte de una comprensión incompleta del problema. Por eso,

analizar, preguntar y validar antes de diseñar también forma parte del trabajo del ingeniero.

Referencias

● [1] International Institute of Business Analysis. The Business Analysis Standard. Toronto:

International Institute of Business Analysis, 2022.

● [2] International Institute of Business Analysis. IIBA Global Business Analysis Core

Standard. Toronto: International Institute of Business Analysis, 2021.

● [3] Bragilovski, Maxim, Ashley T. van Can, Fabiano Dalpiaz, y Arnon Sturm.

“Leveraging Machines to Derive Domain Models from User Stories.” Requirements

Engineering 30, no. 2 (2025).

ARTÍCULO · ESTUDIANTE

De la visión al código: estrategias para una gestión efectiva de requerimientos

Autores: Elian Angel Fernando Reyes Yac; Ramiro Agustín Télles Carcuz

Palabras clave: Arquitectura de software, ingeniería de requerimientos, requerimientos funcionales, requerimientos no funcionales.

Introducción

La ingeniería de software moderna enfrenta el enorme desafío de traducir complejas expectativas humanas en especificaciones sistémicas que resulten verificables. Este artículo analiza la ingeniería de requerimientos para el plasmado de los requerimientos del usuario, se clasifican los distintos tipos de requerimientos existentes, así como, siete funciones en la ingeniería de requerimientos propuestas por Pressman para la elicitación de requerimientos.

Se analiza el uso de maquetas visuales para la validación de requerimientos debido a que puede llegar a ser una herramienta importante para el entendimiento del sistema y la clarificación de requerimientos, esto también previene la implementación errónea de requerimientos. De esta manera, se busca establecer un marco de trabajo que optimice la comunicación entre los equipos de desarrollo y los usuarios finales.

Desarrollo

La ingeniería de requerimientos se centra en todas las actividades referentes a identificar y documentar las necesidades de los clientes. Para este fin aplica principios científicos y técnicas para desarrollar, comunicar y gestionar los requerimientos del usuario (Medina Cruz, Pineda Ballesteros, and Téllez Acuña, 2019).

La International Institute of Business Analysis define los requisitos como: “Los Requerimientos Incluyen, pero no están limitados a, a condiciones del pasado, presente o futuro o capacidades en una empresa y descripciones de la estructura organizacional, roles procesos, políticas, reglas, e información del sistema. Un requerimiento debe describir el estado actual o futuro de cualquier aspecto de la empresa” (Wong Durand, 2017). Según Pressman (WONG DURAND, 2017), la ingeniería de requisitos comprende siete funciones que permiten identificar, definir, validar y gestionar las necesidades de un proyecto de software. Estas funciones, que abarcan desde el inicio del proyecto hasta la gestión de los requisitos, se presentan en la Figura 1.

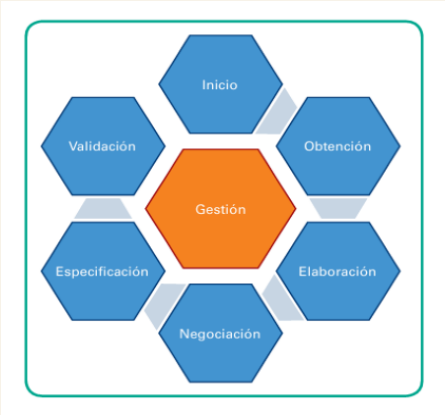


Figura 1. Funciones de la Ingeniería de Requisitos según Pressman. Fuente: Wong Durand (2017).

Como ejemplo, en el posible desarrollo de una herramienta educativa para el aprendizaje de Código SQL Oracle, primero se identificarían las necesidades de los estudiantes y docentes, determinando qué problemas se busca resolver. Posteriormente, se establecerían los límites del proyecto y los requisitos principales, como la ejecución de consultas SQL, la práctica de normalización y la elaboración de diagramas entidad-relación.

Con toda esta información se analizarían aspectos como el tiempo y los recursos disponibles, se elaborarían prototipos para comprobar que la propuesta responde a las necesidades planteadas y, finalmente, los requisitos serían validados y registrados para controlar los cambios que puedan surgir durante el desarrollo.

El uso complementario de maquetas visuales sirve como un potente mecanismo de validación e iteración constante entre los ingenieros y los clientes. Al utilizar maquetas funcionales de manera temprana se clarifica el entendimiento del sistema. Esta técnica permite prevenir la implementación errónea de funciones no evaluadas correctamente. Las consideraciones previas y el diseño de pantallas son fundamentales para garantizar que la

herramienta digital proporciona una usabilidad sobresaliente y satisfactoria. Un software bien

diseñado técnica y visualmente reduce en gran medida las frustraciones en el uso cotidiano.

Conclusiones

La transformación efectiva de necesidades humanas en especificaciones técnicas requiere la

aplicación rigurosa de metodologías de ingeniería, diseñadas específicamente para minimizar la

incertidumbre y reducir drásticamente la frustración del usuario final durante la operación del

sistema. Es necesario realizar un análisis documental exhaustivo, integrando múltiples

perspectivas orientadas al usuario, lo cual facilita la construcción de un panorama integral.

Convertir las necesidades subjetivas de los usuarios en métricas medibles y cuantificables resulta

un paso absolutamente crucial para lograr definir el alcance de los proyectos informáticos de una

manera realista, coherente y profesional. En definitiva, los patrones de interacción y la

experiencia del usuario constituyen los pilares fundamentales que moldean tanto la solución

general como la escalabilidad futura de la arquitectura, garantizando así el éxito comercial.

Referencias

Arciniegas, José L., Verónica Fernández, Amparo Hormiga, Aleyda Tulande, Fernando A

Urbano, and César A Collazos. 2009. “Proceso de requerimiento y análisis para la

definición de la arquitectura desde la perspectiva de usabilidad para el desarrollo de

aplicaciones en la Web.” Avances en Sistemas e Informática 6, no. 2 (Septiembre):

205-210. http://www.redalyc.org/articulo.oa?id=13 3113598023.

Medina Cruz, Javier, Eliécer Pineda Ballesteros, y Freddy Reynaldo Téllez Acuña. 2019.

“Requerimientos de software: prototipado, software heredado y análisis de documentos.”

Ingeniería y Desarrollo 37 (2): 327–345. https://doi.org/10.14482/inde.37.2.1053.

Wong Durand, Sandra. 2017. Análisis y requerimientos de software. Huancayo, Perú:

Universidad Continental.

https://hopelchen.tecnm.mx/principal/sylabus/fpdb /recursos/r138017.PDF.

ARTÍCULO · ESTUDIANTE

La arquitectura que nadie pidió: cuando las decisiones técnicas se alejan de las necesidades del usuario

Autores: María Patricia Serrano Ramírez; Adler Alejandro Pérez Asensio

Palabras clave: Arquitectura de software, Decisiones arquitectónicas, Requisitos de software, Sobreingeniería, Atributos de calidad.

Introducción

¿De qué sirve construir una arquitectura técnicamente impecable si nunca fue necesaria para

resolver el problema? En el desarrollo de software, una decisión arquitectónica debe surgir

siempre de las necesidades y las restricciones que definen el sistema. Ignorar esto puede

convertir lo que era una solución sencilla en una compleja generando un aumento de costos y

esfuerzo.

Por ello, los requisitos funcionales y los atributos de calidad deben construir la base para

determinar qué arquitectura es adecuada para el sistema. El verdadero desafío al diseñar un

sistema no es generar la arquitectura más sofisticada sino lograr justificar aquella que responda

de manera efectiva al problema.

De la necesidad a la decisión arquitectónica

¿Qué pasaría si la arquitectura de un sistema fuera la respuesta a una pregunta que nunca se

hizo? En el desarrollo de software, el adoptar tecnologías antes de lograr comprender

correctamente el problema hace que una solución sencilla se vuelva una estructura

innecesariamente compleja. Por ello, las decisiones deben ser fundamentadas por los requisitos

del sistema y no por la popularidad de las tecnologías.

Tanto los requisitos funcionales como los atributos de calidad, rendimiento, seguridad,

disponibilidad y escalabilidad deben orientar las decisiones de arquitectura (Lago, Avgeriou, and

Hilliard 2010). Cada componente que se incorpora al sistema debe responder a una necesidad

concreta y debe aportar un beneficio que se pueda justificar.

Cuando la arquitectura supera al problema

La Sobre Ingeniería ocurre cuando la solución técnica que se plantea es mucho más compleja del

problema a resolver. Un ejemplo es adoptar una arquitectura distribuida para una aplicación

sencilla. Aunque estas decisiones pueden parecer preparación futura, genera una consecuencia

medible al chocar con los atributos de calidad, disparando costos de mantenimiento, pruebas y

dificultades que no existían sin un beneficio que lo justifique (Kazman, Klein, and Clements

2000).

Esto no significa que la complejidad sea negativa ya que hay sistemas que si lo necesitan como

los sistemas que procesan datos sensibles o las plataformas con miles de usuarios. En estos

casos, estas arquitecturas generan beneficios reales. El problema surge cuando una solución

compleja se adopta sin tener un requisito que la justifique. La complejidad deja de ser una

inversión cuando no se responde a las necesidades reales del sistema convirtiéndose en una

complejidad accidental, introducida por tendencias, imitaciones o expectativas poco realistas.

Un mismo problema, diferentes soluciones

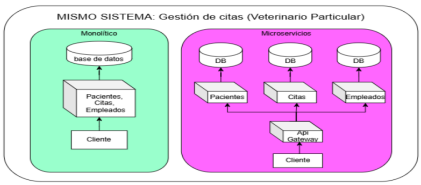
Bien, para verlo de mejor manera se plantea el caso: “Desarrollo de sistema para la gestión de

citas de un veterinario particular”, contemplando registrar usuarios, agendar cita, consultar

disponibilidad, cancelar cita, entre otras. Para darle solución al problema veremos dos

posibilidades, una de ella utilizando un monolítico y una arquitectura basada en microservicios.

mejor manera se plantea el caso: “Desarrollo de sistem
ıario particular”, contemplando registrar usuarios, age
celar cita, entre otras. Para darle solución al prob
e ella utilizando un monolítico y una arquitectura basada



ión de una arquitectura monolítica y una arquitectura basada en
un mismo sistema de gestión. Fuente: Elaboración propia.

nta la Figura 1, una arquitectura basada en microser

Figura 1. Comparación de una arquitectura monolítica y una arquitectura basada en microservicios para un mismo sistema de gestión. Fuente: elaboración propia.

Tal como lo presenta la Figura 1, una arquitectura basada en microservicios representaría

aproximadamente el triple de despliegues a mantener, pues se observan 3 servicios, que precisamente en el caso planteado, cuentan con un sistema de persistencia cada uno, frente a una

aplicación global y una base de datos que representaría un monolítico. Ojo, que no

automáticamente significa que está mal, pero estamos hablando de un “Sistema para la gestión

de citas de un veterinario particular”, no hay ningún driver, como alta disponibilidad o

escalamiento masivo que justifique la implementación de un proyecto de tal magnitud.

Conclusión

Para diseñar una arquitectura es importante tener en cuenta que no se construye un sistema para

relucir conocimientos técnicos, o para dar a conocer habilidades. Esos dos factores son

esenciales, pero no son la prioridad; la prioridad en todo momento es buscar la mejor solución al

problema inicial y satisfacer las necesidades del usuario. Por supuesto, la propuesta tiene que

tener fundamento técnico y tiene que ser hecha a la medida, y en esta parte es donde se tiene que

demostrar cuán capacitado está un arquitecto, porque esto no va de suponer que estilo

arquitectónico es mejor valorado, o más difícil de implementar, consiste en buscar una solución

apta. Basar la arquitectura en la definición de “architecture drivers” es una práctica

indispensable, pues estos ponen restricciones, límites, y alcances que condicionan la elaboración

y planificación de un proyecto, esta práctica ayuda a que el producto final sea una solución real.

Referencias bibliográficas

1. Lago, Patricia, Paris Avgeriou, and Rich Hilliard. 2010. Guest Editors’ Introduction:

Software Architecture: Framing Stakeholders’ Concerns, IEEE Software 27 (6).

https://pure.rug.nl/ws/files/2632678/2010IEEESoftwareLago.pdf.

2. Kazman, Rick, Mark Klein, and Paul Clements. 2000. ATAM: Method for Architecture

Evaluation, CMU/SEI-2000-TR-004. Pittsburgh, PA: Software Engineering Institute,

Carnegie Mellon University.

https://www.sei.cmu.edu/documents/629/2000_005_001_13706.pdf.

3. Gonzalez, Rodrigo Alejandro, Sergio Agustín Giménez, Numa Roy Molina Pualuk, and

Rodrigo Zalazar. 2024. “¿Son Los Microservicios La Mejor opción? Una evaluación De

Su Eficacia Y Eficiencia Frente a Los Monolitos”. JAIIO, Jornadas Argentinas De

Informática 10 (5). https://revistas.unlp.edu.ar/JAIIO/article/view/17980.

ARTÍCULO · ESTUDIANTE

Reaccionar a tiempo: la diferencia entre ajustar un análisis y rehacer la arquitectura

Autores: José Luis Espinoza Jolón; José David Góngora Olmedo

Palabras clave: Análisis funcional, Arquitectura de software, Historias de usuario, Atributos de calidad, Detección temprana.

Introducción

En todos los proyectos de software surge la tentación de avanzar lo más rápido que se pueda al diseño y desarrollo del software, dejando la parte de análisis de requerimientos como un paso más o solo por “compromiso”. Pero un malentendido de negocio que no se detecta a tiempo no desaparece, sino que se traspasa a la arquitectura. Lo que se sentía como tiempo ahorrado termina siendo el origen de un problema más costoso de resolver.

Este artículo surge de una idea simple, pero algo ignorada: ajustar un análisis funcional cuesta minutos, en cambio, rehacer una arquitectura ya construida puede costar semanas de trabajo.

Teniendo en cuenta esa diferencia, se busca entender por qué el costo de un error crece con el paso del tiempo y demostrar que lo importante no es un análisis perfecto, sino la capacidad de reaccionar a tiempo.

Desarrollo

Por qué el costo de un error crece con el tiempo

El costo de corregir un error de requerimientos aumenta drásticamente mientras más tiempo pasa desde el momento en el que se comete y hasta que se detecta (Boehm 1981). Un malentendido detectado en análisis se corrige de inmediato; el mismo malentendido, detectado ya con la arquitectura construida, puede implicar rediseñar componentes completos.

Aun así, esto no se debe tomar como una regla absoluta. Las proporciones exactas dependen del tamaño y alcance del proyecto. Estas también indican que fijar los requerimientos de manera muy anticipada para evitar cambios a última hora puede traer consecuencias, una de ellas: el construir de manera precisa algo que el usuario ni siquiera necesitaba (Boehm y Basili 2001). Esto lleva a considerar que lo relevante no es evitar los cambios lo más que se pueda, sino desarrollar la habilidad de detectar y corregir los problemas lo más rápido posible, antes de que la arquitectura sea construida sobre un error.

Un caso simple es: un cliente pide ‘un sistema que gestione pedidos’ sin aclarar si incluye facturación o notificaciones. Detectar esa ambigüedad en análisis cuesta una pregunta; detectarla con el módulo ya construido implica rediseñarlo por completo.

Ilustración 1 Gráfica ilustrativa de la tendencia en el costo de corregir un error según la etapa del proyecto



Ilustración 1. Gráfica ilustrativa de la tendencia en el costo de corregir un error según la etapa del proyecto. Fuente: elaboración propia con Excel, basada en el concepto de Boehm (1981).

Cómo se ve esto en la práctica

Para entender de mejor manera como crece el costo de un error, debemos analizar qué sucede cuando un error pasa desapercibido durante el análisis. Si se detecta una mala interpretación a tiempo, la corrección se limita a un documento, sin embargo, si escala al nivel de arquitectura, una simple aclaración se convierte en semanas de trabajo desperdiciado.

Un arreglo sencillo puede realizarse en las historias de usuario y los casos de uso. Un ejemplo ayuda a ver esta diferencia con claridad. En un sistema de aprobación de pagos, el requisito original decía algo así: "el sistema debe aprobar los pagos de forma rápida y segura". Esta redacción no dice nada verificable: ¿qué tan rápido es rápido?, ¿qué reglas definen que un pago sea seguro? Durante el análisis, el equipo lo rehace como: "aprobar automáticamente los pagos menores a Q5,000 en un máximo de 3 segundos, siempre que el usuario tenga historial sin incidencias en los últimos 6 meses; los pagos mayores requieren validación manual". Como señalan Wiegers y Beatty (2013), modificar un requisito en papel evita que

los errores se transmitan al código, lo que permite realizar ajustes sin agotar el presupuesto.

Errores de diseño que afectan atributos de calidad (escalabilidad o modelado de datos)

detectados en producción requieren intervenciones severas, como reestructurar la base de

datos o rediseñar componentes clave. La prioridad técnica debe centrarse en la detección

temprana mediante análisis de requerimientos, no en la corrección reactiva.

Este mismo criterio se aplica al uso de IA en el análisis funcional, ya que puede acelerar la

redacción de una historia de usuario o sugerir un primer requisito, pero no puede decidir qué

es ambiguo ni interpretar el contexto del negocio. Esta validación sigue siendo

responsabilidad del analista.

Conclusión

Lo que separa a un proyecto exitoso de uno costoso no es contar con un análisis perfecto,

sino la capacidad de reaccionar a tiempo ante un error. Como muestran Boehm (1981) y

Boehm y Basili (2001), el costo de una corrección no depende de la complejidad del error,

sino del momento en que se detecta: mientras siga siendo una idea en un documento, ajustarla

cuesta minutos; una vez que se traslada a la arquitectura, cuesta semanas de rediseño.

El análisis funcional no es un trámite que retrasa el desarrollo, sino la herramienta que

permite corregir el rumbo mientras todavía es barato hacerlo (Wiegers y Beatty 2013). Una

arquitectura sólida no depende de la suerte ni de programar rápido, sino de cuestionar cada

requisito ambiguo antes de construir sobre él. La diferencia entre ajustar y rehacer se decide

mucho antes de escribir la primera línea de código.

Referencias

Boehm, Barry W. 1981. Software Engineering Economics. New Jersey: Prentice-Hall.

Boehm, Barry, y Victor R. Basili. 2001. “Software Defect Reduction Top 10 List”.

Computer 34, n. 1: 135-137. https://ieeexplore.ieee.org/document/962984

Wiegers, Karl, y Joy Beatty. 2013. Software Requirements. 3.ª ed. Redmond, WA:

Microsoft Press

ARTÍCULO · ESTUDIANTE

Arquitecto de la torre de marfil (ITA): dificultades del papel a la implementación

Autores: Marco Fernando Cruz Mendoza; Erick Estuardo Muñoz Escalante

Palabras clave: Arquitectura de software, torre de marfil, erosión arquitectónica, desarrollo de software, toma de decisiones, ITA.

Introducción:

La arquitectura de software permite definir la estructura de un sistema y orientar las

decisiones necesarias para su desarrollo. Sin embargo, cuando estas se toman ignorando la

realidad del proyecto y con poca comunicación con el equipo encargado de implementarlas,

puede surgir una brecha entre el diseño y la construcción. Esta situación se puede relacionar

con la problemática del arquitecto de la torre de marfil.

El presente artículo explora las dificultades que surgen cuando las decisiones arquitectónicas

se alejan de las condiciones reales del desarrollo. También busca destacar la importancia de la

comunicación y colaboración entre arquitectos y desarrolladores de software, para favorecer

la retroalimentación y considerar las diferentes perspectivas sobre el proyecto.

Desarrollo:

La arquitectura de software cumple una función importante en el desarrollo de software,

debido a que establece la estructura y la comunicación entre los distintos componentes que

conforman un sistema. Sin embargo, surge la siguiente interrogante: ¿Qué pasaría si el

arquitecto encargado se apega a reglas y estándares «irreales» para establecer el modelo de

desarrollo que se utilizará en la construcción del software?

La problemática de la torre de marfil es una metáfora utilizada para hacer referencia a una

posición de aislamiento, asociada con figuras que desempeñan un rol de liderazgo. Como se

ilustra en la Figura 1, este aislamiento puede favorecer una cadena de desconexiones que

culmina en la erosión arquitectónica del sistema. En contraste, la comunicación constante y la

colaboración entre arquitectos y desarrolladores pueden favorecer una arquitectura más

adaptable y alineada con las necesidades reales del proyecto.

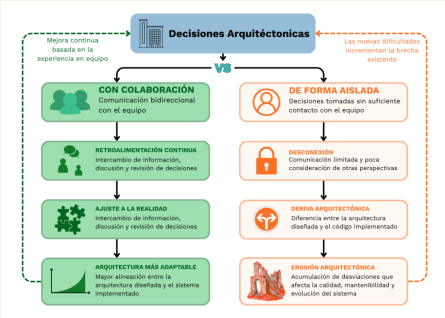


Figura 1. Comparación de los efectos de la toma de decisiones aislada y colaborativa en la arquitectura de software. Fuente: elaboración propia.

Debido al rol de liderazgo del arquitecto de software, puede presentarse la problemática de la

torre de marfil cuando sus decisiones se alejan de las herramientas, los procesos y las

necesidades reales del equipo. Esta desconexión puede dificultar la implementación de la

arquitectura propuesta y favorecer diferencias entre el diseño y el sistema construido.

Un ejemplo puede presentarse cuando un arquitecto decide implementar una solución cloud

ready basada en microservicios, Redis y Kubernetes en un sistema on-premise altamente

restrictivo. Desde el punto de vista administrativo, el diagrama de arquitectura puede parecer

lógico e impecable. Sin embargo, para el equipo de desarrollo la problemática no se resuelve,

sino que se agrava, debido a que el diseño asume conectividad permanente y alta

disponibilidad en la nube, ignorando las restricciones del negocio y la naturaleza offline de

los sistemas de facturación.

En este caso, la disponibilidad y la escalabilidad constituyen atributos de calidad que orientan

la decisión arquitectónica, pero implican un trade-off frente a las restricciones de

conectividad y las necesidades operativas del sistema. Entonces, ¿cómo pueden atenuarse los

efectos negativos asociados con la problemática de la torre de marfil en la arquitectura de software? Según Emmanuel Valverde Ramos, desde una perspectiva de equipo, es posible

reducir estos efectos mediante las siguientes estrategias:

● Mantenerlo sencillo: utilizar explicaciones claras y sencillas.

● Reuniones casuales: realizar reuniones en un contexto de intercambio de ideas.

● Registrar tiempos entre ciclos: obtener datos relevantes sobre deficiencias.

Conclusiones:

La problemática del arquitecto de torre de marfil demuestra que una arquitectura no debe

evaluarse únicamente por su calidad en el diseño, sino también por su capacidad para

adaptarse a las condiciones reales de su implementación. Cuando no existe una alineación

entre los objetivos de ambas perspectivas se pueden generar diferencias entre la estructura

planeada y el producto final, dando como resultado problemas como la erosión

arquitectónica. Como recomendación, debe existir una revisión y comunicación frecuente

entre el arquitecto y los desarrolladores de software, de manera que la toma de decisiones se

pueda contrastar con la realidad del proyecto. Es importante encontrar un equilibrio entre

ambas perspectivas, de modo que la visión del arquitecto tampoco se vea limitada, para que

se puedan incorporar soluciones con capacidad de evolución, pero sin agregar complejidad

innecesaria.

Referencias bibliográficas:

Baabad, A., Zulzalil, H., Hassan, S., & Baharom, S. Characterizing the Architectural Erosion

Metrics: A Systematic Mapping Study. IEEE Access, 10 (2022): 22915–22940.

https://doi.org/10.1109/ACCESS.2022.3150847

Uzun, B., & Tekinerdogan, B. Detecting deviations in the code using architecture view-based

drift analysis. Computer Standards & Interfaces, 87 (2024): 103774.

https://doi.org/10.1016/j.csi.2023.103774.

Alami, A., Krancher, O., & Paasivaara, M. The journey to technical excellence in agile

software development. Information and Software Technology, 150 (2022): 106959.

https://doi.org/10.1016/j.infsof.2022.106959

Valverde Ramos, E. The Ivory Tower Architect. The Tech Lead Journal, Substack (2024).

https://emmanuelvalverderamos.substack.com/p/the-ivory-tower-architect

ARTÍCULO · ESTUDIANTE

Comunicación efectiva entre analistas, arquitectos y programadores

Autores: Jens Jeremy Pablo Sosof; Kevin Estuardo Sotoj García

Palabras clave: Comunicación, Requisitos, Código, Arquitectura de Software

Introducción

En el mundo moderno, los sistemas de software se han convertido en la columna vertebral de

muchos sectores. Procesos manuales muy complejos, hoy se resuelven en menos tiempo por

medio de plataformas digitales. Esto es posible por la ingeniería de software, ya que dividió

el desarrollo en roles especializados: analista, arquitecto y programador.

La comunicación entre estos tres roles no es un aspecto secundario ya que determina

directamente el éxito o fracaso del desarrollo de software. La inteligencia artificial puede

automatizar parte de la traducción entre necesidades del negocio y las especificaciones

técnicas. No obstante, la validación final debe permanecer en manos del analista humano.

Desarrollo

La comunicación en el desarrollo de software traduce necesidades humanas en código

funcional a través de especificaciones y diagramas. Estas herramientas codifican las reglas de

negocio y relaciones lógicas que el equipo de programación debe implementar.1

Una comunicación clara produce software robusto, mantenible y alineado con el presupuesto

y tiempos del negocio. Este flujo entre roles opera en tres fases:

1. Especificación y Modelado: Levantamiento de requisitos y reglas de negocio

detalladas por parte del analista para sentar las bases funcionales del sistema.

2. Diseño de Arquitectura y Contratos: Traducción de los requisitos funcionales en

diagramas estructurales, patrones de diseño y contratos de API que guían la fase de

codificación. 3. Codificación, Evaluación y Ajuste: Implementación del código, retroalimentación

técnica continua sobre las restricciones reales encontradas en el desarrollo, y ajuste

dinámico de las especificaciones funcionales y de diseño.

Según The Chaos Report de Standish Group y estudios como el de Rashaad R. Randall, las

brechas de comunicación entre los equipos de negocio hacen que el 80% de los proyectos

excedan presupuestos o plazos de entrega.3 Este dato es relevante en equipos pequeños, donde

una sola persona suele asumir a la vez el rol de analista y de arquitecto. En esos casos,

pueden surgir ambigüedades que pasan desapercibidas hasta etapas avanzadas del desarrollo.

Ventajas de una comunicación fluida	Inconvenientes de la brecha comunicativa
Alineación con el negocio: el software satisface necesidades y objetivos.	Desalineación y retrabajo: se construyen características innecesarias por interpretar mal los requisitos.
Mayor eficiencia: reduce ciclos tardíos de corrección y desperdicio.	Acumulación de deuda técnica: atajos de código degradan el sistema.
Mantenibilidad y escalabilidad: el código sigue el diseño planificado.	Desviación arquitectónica: el código final se aleja de la estructura diseñada.

Tabla 1. Ventajas vs. inconvenientes de la comunicación. Fuente: elaboración propia, 2026, a partir de las fuentes bibliográficas consultadas.

tabla comparativa que resume las principales ventajas de una comunicación fluida frente a los

inconvenientes derivados de una brecha comunicativa.

La interacción humana aún presenta graves desafíos.2 Destacan la alucinación de requisitos

como asumir funcionalidades inexistentes, los outputs técnicos incoherentes, APIs que no

respetan los contratos de diseño y la pérdida de traducción por documentación ambigua o la

falta de un lenguaje de dominio común. Si bien las herramientas de inteligencia artificial

actuales pueden guiar, ayudar o realizar parte del trabajo de esta traducción, estas no

reemplazan la validación humana que debe realizar un analista. Por ejemplo, un modelo de inteligencia artificial puede producir una especificación muy completa pero fuera del alcance

real del proyecto. La validación humana es el control final del software desarrollado.

Conclusiones

La comunicación ha sido y sigue siendo un tema de discusión dentro de la ingeniería de

software desde hace décadas. La creciente complejidad de los ecosistemas en la nube, la

adopción de microservicios y herramientas de inteligencia artificial, han impulsado la

necesidad de una alineación sin precedentes entre analistas, arquitectos y programadores. Una

comunicación estructurada y ágil ofrece enormes beneficios en términos de productividad y

calidad para las organizaciones. Aunque coordinar la visión de negocio, el diseño técnico y la

ejecución del código presenta retos humanos y organizacionales reales, las empresas exitosas

continúan explorando cómo las nuevas herramientas de inteligencia de software, el modelado

automático y las plataformas de colaboración en tiempo real pueden refinar sus flujos de

trabajo internos y enriquecer sus productos finales.

Referencias

1. Pedro Pablo González Pérez e Ismael Soto Galindo, "Modelado, arquitectura y

comunicación en el desarrollo de sistemas de software: un enfoque práctico" (libro de texto,

Universidad Autónoma Metropolitana, 2023), consultado el 17 de agosto de 2026

http://ilitia.cua.uam.mx:8080/jspui/bitstream/123456789/1160/1/Modelado%20arquitectura%

20y%20comunicacion%20en%20el%20desarrollo%20de%20sistemas%20de%20software_%

20un%20enfoque%20practico-VF.pdf

2. José Cruz Estefanía Almanza, "Propuesta para reducir la brecha de comunicación mediante

pruebas de software basadas en comportamiento ISO/IEC 29110" (tesis de maestría,

Universidad Autónoma de Querétaro, 2026), consultado el 20 de agosto de 2026,

https://ri-ng.uaq.mx/bitstream/123456789/13079/1/IFMAC-253822.pdf

3. Rashaad R. Randall, "Lost in Translation: The Cost of Communication Gaps", en Insights

on Software Engineering & Leadership, consultado el 20 de agosto de 2026,

https://rashaadrandall.com/insights/lost-in-translation-the-steep-cost-of-software-communicat

ion-gap